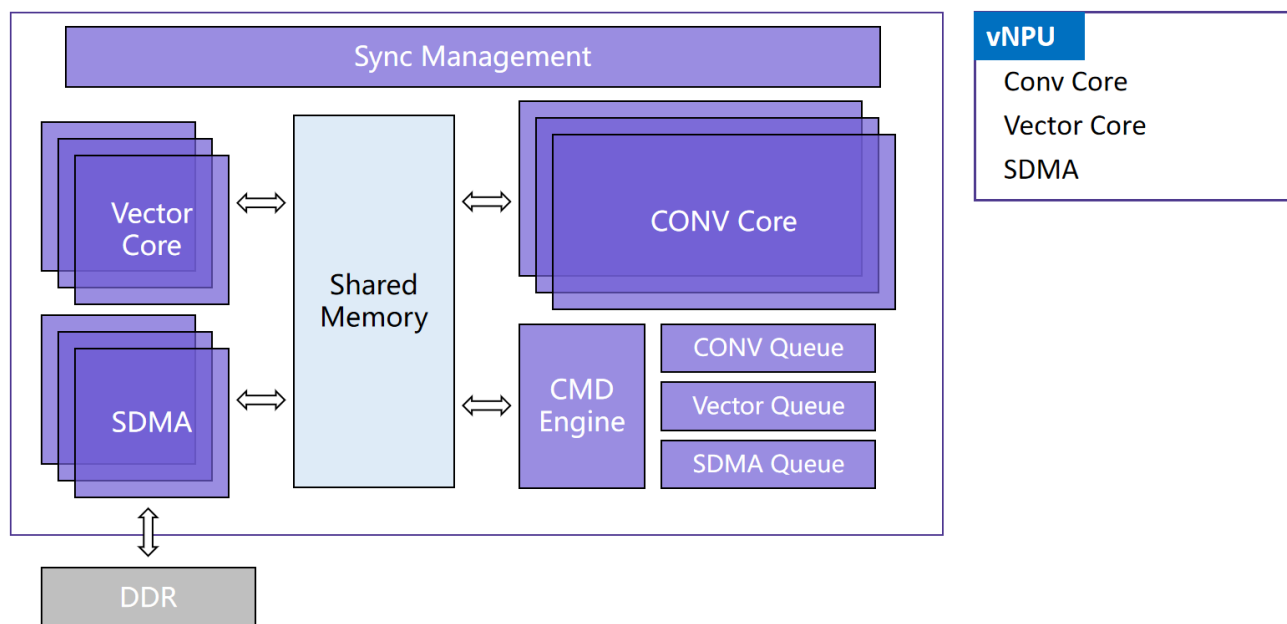


NPU 架构



异构多核架构，以 Tensor 为核心的流计算

- EU
 - CONV / TENG / SDMA
 - RDMA / WDMA / CORE
 - RDMA / WDMA 类似 GPU 的 TMA，有硬件有基于高维 Tensor 抽象进行跳读跳写数据的能力
 - 3 维 Tensor Transpose 读取
 - `a.shape = (12, 2, 32)`
 - `b = a.transpose(0, 2, 1)` # 实际上是免费的，没有真正操作数据

```
cfg.teng.rdma_cfg[0].bytes_max = 31
cfg.teng.rdma_cfg[0].offset_addr = 492576
cfg.teng.rdma_cfg[0].stride[0] = 2112
cfg.teng.rdma_cfg[0].stride[1] = 32
cfg.teng.rdma_cfg[0].stride[2] = 4224
cfg.teng.rdma_cfg[0].line_max[0] = 1
cfg.teng.rdma_cfg[0].line_max[1] = 65
cfg.teng.rdma_cfg[0].line_max[2] = 11
```

- 每个 EU 有独立的指令队列，EU 之间异步执行，通过 SyncManager 进行同步
- Sync Manager

mcode (instruction)

- LDI / TRIGGER (async) / WAIT IDLE / SYNC

682	681	300000000199000A1	LDI	199	<=	300000000	# crc=0
683	682	40000000019A000A1	LDI	19A	<=	400000000	# crc=0
684	683	50000000019B000A1	LDI	19B	<=	500000000	# crc=0
685	684	60000000019C000A1	LDI	19C	<=	600000000	# crc=0
686	685	70000000019D000A1	LDI	19D	<=	700000000	# crc=0
687	686	80000000019E000A1	LDI	19E	<=	800000000	# crc=0
688	687	90000000019F000A1	LDI	19F	<=	900000000	# crc=0
689	688	A000000001A0000A1	LDI	1A0	<=	A00000000	# crc=0
690	689	B000000001A1000A1	LDI	1A1	<=	B00000000	# crc=0
691	690	C000000001A2000A1	LDI	1A2	<=	C00000000	# crc=0
692	691	D000000001A3000A1	LDI	1A3	<=	D00000000	# crc=0
693	692	E000000001A4000A1	LDI	1A4	<=	E00000000	# crc=0
694	693	F000000001A5000A1	LDI	1A5	<=	F00000000	# crc=0
695	694	0000FFFF185000A1	LDI	185	<=	0000FFFF	# crc=0
696	695	000000001015000A1	LDI	015	<=	000000001	# crc=0
697	696	000000000072000A8	MV	072	<=	000000000	# crc=0
698	697	005D5480073000A1	LDI	073	<=	005D5480	# crc=0
699	698	000000008078000A1	LDI	078	<=	000000008	# crc=0
700	699	001000000015000A1	LDI	015	<=	001000000	# crc=0
701	700	000000000000000A3	WAIT	000	<=	000000000	# crc=0
702	701	010000000015000A1	LDI	015	<=	010000000	# crc=0
703	702	000000000000000A9	YIELD	000	<=	000000000	# crc=0
704	703	003000053000000A2	SYNC WAIT 03	<=	5		
705	704	004000053000000A2	SYNC WAIT 04	<=	5		
706	705	005000053000000A2	SYNC WAIT 05	<=	5		

job / register config

- 对硬件进行配置，触发硬件执行，等待硬件执行完成的一组 mcode 集合
- 与硬件架构直接相关

```
"cfg.teng.enable.trans_en = 1
cfg.teng.idle_mask_cfg.wait_compute = 1
cfg.teng.idle_mask_cfg.wait_rdma[0] = 1
cfg.teng.idle_mask_cfg.wait_wdma[0] = 1
cfg.teng.main_csb_cfg.mux_input = 2
cfg.teng.main_csb_cfg.mux_reshape = 1
cfg.teng.rdma_cfg[0].bytes_max = 31
cfg.teng.rdma_cfg[0].offset_addr = 169248
cfg.teng.rdma_cfg[0].stride[0] = 2112
cfg.teng.rdma_cfg[0].stride[1] = 32
cfg.teng.rdma_cfg[0].stride[2] = 4224
cfg.teng.rdma_cfg[0].line_max[0] = 1
cfg.teng.rdma_cfg[0].line_max[1] = 65
cfg.teng.rdma_cfg[0].line_max[2] = 15
cfg.teng.rdma_cfg[0].jump_en = 1
cfg.teng.rdma_cfg[0].jump_thr_32B[0] = 5817
cfg.teng.rdma_cfg[0].jump_thr_32B[1] = 6345
cfg.teng.rdma_cfg[0].jump_thr_32B[2] = 6873
cfg.teng.rdma_cfg[0].jump_skip_32B[0] = 990
cfg.teng.rdma_cfg[0].jump_skip_32B[1] = 1624
cfg.teng.rdma_cfg[0].jump_skip_32B[2] = 1624
cfg.teng.reshape_csb_cfg.mux_input = 4
cfg.teng.reshape_csb_cfg.mux_trans = 1
cfg.teng.start_cfg.compute_start = 1
cfg.teng.start_cfg.rdma_start[0] = 1
cfg.teng.start_cfg.wdma_start[0] = 1
cfg.teng.trans_cfg.pad_input_cycle_max = 2111
cfg.teng.trans_cfg.crop_input_cycle_max = 2111
cfg.teng.trans_cfg.sub_trans[0].input_data_width = 2
cfg.teng.trans_cfg.sub_trans[0].en = 1
cfg.teng.wdma_cfg[0].bytes_max = 67583
cfg.teng.wdma_cfg[0].offset_addr = 298272
cfg.teng.wdma_cfg[0].jump_en = 1
cfg.teng.wdma_cfg[0].jump_thr_32B[0] = 9849
cfg.teng.wdma_cfg[0].jump_thr_32B[1] = 10377
cfg.teng.wdma_cfg[0].jump_thr_32B[2] = 10905"
```

operator

- 算子实现：将算法语义算子映射为一系列硬件执行任务 (job) 的过程
- 基础算子
 - 基础算子定义尽可能稳定，语义接近硬件实际的计算过程
 - 算子实现直接采用 job 编程，不同代 NPU 之间不兼容
 - elemwise_int / elemwise_float / fma_int / fma_float / reduce_int / reduce_float / transpose / pad ...
- 高层算子
 - 高层算子定义与开源框架 (torch) 基本对齐，进而保证模型算法精度对齐
 - 基于基础算子构建计算图实现计算过程，确保在不同代 NPU 之间的兼容性
 - quant_axmodel 中的各类 Ax 开头算子

AI-ISP 模型编译过程及中间产物

- graph partition
 - 按照配置文件修改模型分辨率，切分 partition
 - [partition.onnx](#)
- graph schedule (slice mode)
 - 对每个 partition 进行调度，根据配置文件对输出进行行切分，逐层反向推理直至输入，由 partition 维度的计算图变为 slice 维度的计算图
 - output/compiler/debug/isp_model/sched_slice_intervals_\${partition_id}.log

```

27 20 INTERVAL EPOCH slice -230:-224
28 27 INTERVAL EPOCH slice -224:-192
29 28 INTERVAL COMPUTE OP 1 WRITES op_a43e9afe:AxSpace2Depth_out_0[0:4] READS current_nhwc_0[0:8] ESTIMATED 0 MACS
30 29 INTERVAL COMPUTE OP 2 WRITES op_e7bf1829:AxQuantizedSub_out_0[0:4] READS op_a43e9afe:AxSpace2Depth_out_0[0:4] AND offset_in[...] ESTIMATED 0 MACS
31 30 INTERVAL COMPUTE OP 3 WRITES op_39506388:AxResize_out_0[0:2] READS op_e7bf1829:AxQuantizedSub_out_0[0:4] ESTIMATED 0 MACS
32 31 INTERVAL COMPUTE OP 28 WRITES op_e4c1a9b3:AxQuantizedConv_out_0[0:1] READS op_39506388:AxResize_out_0[-1:2] ESTIMATED 76,032 MACS
33 32 INTERVAL COMPUTE OP 1 WRITES op_a43e9afe:AxSpace2Depth_out_0[4:6] READS current_nhwc_0[8:12] ESTIMATED 0 MACS
34 33 INTERVAL COMPUTE OP 2 WRITES op_e7bf1829:AxQuantizedSub_out_0[4:6] READS op_a43e9afe:AxSpace2Depth_out_0[4:6] AND offset_in[...] ESTIMATED 0 MACS
35 34 INTERVAL COMPUTE OP 3 WRITES op_39506388:AxResize_out_0[2:3] READS op_e7bf1829:AxQuantizedSub_out_0[4:6] ESTIMATED 0 MACS
36 35 INTERVAL COMPUTE OP 29 WRITES op_e5c555a6:AxQuantizedConv_out_0[0:2] READS op_39506388:AxResize_out_0[-1:3] ESTIMATED 304,128 MACS
37 36 INTERVAL COMPUTE OP 30 WRITES op_6205425f:AxQuantizedConv_out_0[0:1] READS op_e5c555a6:AxQuantizedConv_out_0[-1:2] ESTIMATED 304,128 MACS
38 37 INTERVAL EPOCH slice -192:-160
39 38 INTERVAL COMPUTE OP 1 WRITES op_a43e9afe:AxSpace2Depth_out_0[6:12] READS current_nhwc_0[12:24] ESTIMATED 0 MACS
40 39 INTERVAL COMPUTE OP 2 WRITES op_e7bf1829:AxQuantizedSub_out_0[6:12] READS op_a43e9afe:AxSpace2Depth_out_0[6:12] AND offset_in[...] ESTIMATED 0 MACS
41 40 INTERVAL COMPUTE OP 3 WRITES op_39506388:AxResize_out_0[3:6] READS op_e7bf1829:AxQuantizedSub_out_0[6:12] ESTIMATED 0 MACS
42 41 INTERVAL COMPUTE OP 28 WRITES op_e4c1a9b3:AxQuantizedConv_out_0[1:3] READS op_39506388:AxResize_out_0[1:6] ESTIMATED 152,064 MACS
43 42 INTERVAL COMPUTE OP 1 WRITES op_a43e9afe:AxSpace2Depth_out_0[12:14] READS current_nhwc_0[24:28] ESTIMATED 0 MACS

```

- operator build
 - 将 slice 维度计算图中的各个算子映射为一系列 job，并为所有 job 生成互相之间的依赖关系 (类计算图)
- ocm / eu allocation
 - 根据 job 的输入 tensor、输出 tensor、预期 cycle 数以及 job 之间的依赖关系，为每个 tensor 分配 ocm 空间，为每个 job 分配 eu
 - [trace_schedule.json](#)
- assemble
 - 将 job 转换为 mcode
 - 根据 job 间依赖关系插入 SyncManager 同步指令
 - [trace.json](#)
- 性能分析
 - 算子计算效率
 - 输入数据量 / cycle 数
 - OCM 使用效率